Structure of Computer Systems

Benchmark for mobile devices (Android)

Faculty: Technical University of Cluj-Napoca

Specialization: CTI

Academic year: 2025 – 2026

Student: Simu Darius-Marian

Group: 30435

Professor: Sebestyen-Pal Gheorghe

TABLE OF CONTENTS

1. Introduction 4
   1.1. Project Proposal 4
   1.2. General Overview 4
   1.3. Project Plan 4
2. Bibliographic Study 5
3. Analysis 6
   3.1. Objective and Scope 6
   3.2. Monitored Performance Parameters 6
      I. CPU 6
      II. GPU 6
      III. Memory 6
      IV. I/O (Storage) 6
   3.3. Factors Influencing Performance Variation 6
   3.4. Conceptual Approach to Measurement Methodology 7
      I. Isolation and Environment Control 7
      II. Warm-up 7
      III. Repetition and Sampling 7
      IV. Anomaly detection 7
   3.5. Conceptual Application Pipeline 7
   3.6. Algorithms Used For Implementation 7
      I. Test Results Storage module – saves the result of the tests performed 7
      II. Scoring module – transforms raw data to a generalized score 8
      III. CPU single-core – intensive calculations 9
      IV. CPU multi-core – measures aggregate performance 9
      V. GPU – rendering of a fixed 3D scene 10
      VI. Memory – measures the bandwidth 11
      VII. I/O (Storage) – random and sequential throughput measuring 12
4. Design 13
   4.1 Overview 13

4.2 Use of Theoretical Concepts 13
4.3 General System Architecture 13
4.4 UML Architecture 14
I. User Actions 14
II. Structure of classes 15
4.5 Components General Description 15
I. Controller 15
II. CPU module 15
III. GPU module 15
IV. Memory module 15
V. I/O Storage module 15
VI. Result Aggregator 16
VII. User Interface 16
4.6 Component Interaction 16
5. Implementation 16
5.1 Implementation Overview 16
5.2 Phase 1: Benchmarking Tests 16
5.3 Phase 2: User Interface 18
6. Testing and Validation 20
7. Conclusions 23
8. References 23

1. INTRODUCTION

1.1. PROJECT PROPOSAL

The goal of the project is to design and implement an Android benchmarking application that can display the hardware and software configuration of the device and allows the user to run specific tests that aim to check the execution time, memory access time, graphics processing time and other such parameters and give an appropriate score.

Benchmarking can be very useful when trying to compare different devices, to see which one achieves the greatest performance in any given task and by what margin, it can also help to see if certain Android updates or different versions of it impact the performance of the device.

In order to differentiate from other such benchmarking applications already available, this project will allow the user to choose what test he wants to run from a list, his device receiving a separate score for each particular test and a main test that takes into account all the other test scores.

1.2. GENERAL OVERVIEW

The application will have 2 main components:

1. **System Information Section**: shows general information about the devices hardware and software such as processor model, number of cores, clock frequency, total and available memory, storage capacity, Android version.
2. **Benchmarking Section**: will contain a list of tests from which the user can select specific ones they want to run. These tests will provide parameters such as CPU execution time, memory access time, graphics rendering time, and give an appropriate score.

After all the tests have been run at least once the application will give an overall score of the performance of the device. This score can be used to compare different devices or the impact of certain software updates on the performance of the device.

The chosen IDE is Android Studio, and the chosen programming languages will be Kotlin and C.

1.3. PROJECT PLAN

1. Week 2 (8 Oct): Project chosen.
2. Week 4 (22 Oct): Project proposal, bibliographic study and bibliography.
3. Week 6 (5 Nov): Analysis and Design of the project
4. Week 8 (19 Nov): Implementation: Benchmarking tests
5. Week 10 (3 Dec): Implementation: User interface
6. Week 12 (17 Dec): Testing, Validation and Conclusions

2. BIBLIOGRAPHIC STUDY

Before starting the development of this project, several Android benchmark applications were analyzed in order to better understand their structure and general approach when it comes to analyzing the performance of an Android device. Examples of such applications include: *Geekbench 6*, *AnTuTu Benchmark* and *3D Mark*.

Geekbench uses practical, everyday scenarios and data sets in order to measure the performance of the given device. It tests the processors single-core and multi-core performance as well as the GPU, as described on the official Geekbench website [1].

Pros:

- Tests real-world scenarios
- Provides results in a short time

Cons:

- Limited GPU testing

AnTuTu Benchmark tests User Experience (UX) beside CPU, GPU, Memory and I/O, stating that this "*can clearly give users an impression to the operation efficiency of Android smart phones*", rather than just testing the value of a devices components – [2].

Pros:

- Tests real-time responsiveness (UX)
- Easy to understand score system

Cons:

- Less transparency, not all workloads are explained in detail

3D Mark specializes in gaming benchmarking, it scans the devices hardware and recommends the adequate tests for the specific configuration of the device. It relates the given score to real-world games by estimating the frame rates that can be expected in popular video games – [3].

Pros:

- Highly accurate GPU tests
- Provides real-time performance metrics like FPS and thermal data

Cons:

- Mainly focused on GPU performance, the testing of the overall system is limited

3. ANALYSIS

3.1. OBJECTIVE AND SCOPE

The purpose of this chapter is to present the theoretical concepts necessary to design and implement an Android benchmark application. We will define the performance parameters monitored, the factors that influence their variation, the conceptual measurement methodology and provide some of the algorithms that will be used in pseudocode form.

3.2. MONITORED PERFORMANCE PARAMETERS

I. CPU

- Single-core throughput – operations per second on a single thread
- Multi-core throughput – aggregate performance on multiple threads/cores

II. GPU

- Frames per second (FPS) – frames rendered per second in a graphic scenario
- Thermal-induced throttling – FPS decrease over time due to temperature increasing

III. Memory

- Bandwidth – MB per second for writing and reading operations
- Latency – average access latency

IV. I/O (Storage)

- Read/Write throughput – MB per second for sequential and random
- IOPS – I/O operations per second

3.3. FACTORS INFLUENCING PERFORMANCE VARIATION

- Thermal throttling – temperature increase leads to decreased CPU/GPU performance
- DVFS (Dynamic voltage and frequency scaling) – CPU/GPU adjust their frequencies based on load and power profile
- Core heterogeneity – workload distribution between fast performance cores and efficient cores can influence results
- Battery level – can influence permitted frequencies for the CPU/GPU
- Android version – can affect execution of Java/Kotlin application execution
- Non-deterministic measurements – due to system variability, repetition and statistical methods are required for accurate results

3.4. Conceptual Approach to Measurement Methodology

I. Isolation and Environment Control

- Restart the phone to minimize background processes.
- Use the same brightness and power profile for all tests.

II. Warm-up

- Execute a warm-up phase to bring the system to a steady state.
- Avoid collecting results during this phase.

III. Repetition and Sampling

- Run each test a fixed number of times to obtain accurate results.
- Collect average and p95, p99 for the GPU test.

IV. Anomaly detection

- Filter runs with obvious background processing and significant performance loss.
- Mark and report conditions (frequencies, temperature, battery level).

3.5. Conceptual Application Pipeline

Figure 3.1 shows the general workflow of the application pipeline.

Figure 3.1

3.6. Algorithms Used For Implementation

I. Test Results Storage module – saves the result of the tests performed

The application includes a lightweight persistence system for storing benchmark history. Each test run is saved as a TestResult object containing timestamps, individual test scores, throughput values, FPS metrics, and a computed overall score (only when all tests are run).

The storage is implemented using SharedPreferences combined with Gson for JSON serialization. Results are stored as a list, automatically keeping only the last five benchmark sessions to avoid excessive storage use.

The system also exposes helper functions to retrieve results, check if any history exists, compute the average overall score, and format timestamps for display. This module ensures the app always has a recent history of benchmark performance without requiring external storage or databases.

```
Algorithm Save_Test_Result(result):
  results = load_results()
  insert result at index 0 of results
  if size(results) > 5:
    remove elements from index 5 to end
  json = Gson.toJson(results)
  write json under key "test_results"

Algorithm Get_Average_Overall_Score():
  results = load_results()
  scores = all result.overallScore that are not null
  if scores is empty:
    return null
  return average(scores)
```

II. Scoring module – transforms raw data to a generalized score

This module receives the raw data returned from the benchmark test functions and computes separate scores for each of the tests that has been run and an overall score that accounts for the device's performance across all areas that have been tested in the current run. The scores range from 0 to 1000, where 0 is the worst possible performance (cannot be achieved during actual real-life testing) and 1000 which represents the best possible performance (this also cannot be achieved during actual real-life testing).

```
Algorithm scoreCpuSingleCore(timeSeconds: Double):
  val baseTime = 2.0
  val score = 1000.0 * (baseTime / timeSeconds)
  return min(1000, max(0, score.toInt()))

Algorithm scoreCpuMultiCore(timeSeconds: Double):
  val baseTime = 2.0
  val score = 1000.0 * (baseTime / timeSeconds)
  return min(1000, max(0, score.toInt()))

Algorithm scoreMemory(bandwidthGBps: Double):
  val maxBandwidth = 50.0
  val score = 1000.0 * (bandwidthGBps / maxBandwidth)
  return min(1000, max(0, score.toInt()))

Algorithm scoreStorage(throughputMBps: Double):
  val maxThroughput = 3000.0
  val score = 1000.0 * (throughputMBps / maxThroughput)
  return min(1000, max(0, score.toInt()))
```

```
Algorithm scoreGpu(averageFps: Double):
    val targetFps = 250.0
    val score = 1000.0 * (averageFps / targetFps)
    return min(1000, max(0, score.toInt()))

Algorithm calculateOverallScore(scores: List<Int>):
    if (scores.isEmpty()) return 0
    return scores.average().toInt()
```

III. CPU single-core – intensive calculations

The single-core benchmark measures CPU arithmetic performance by repeatedly computing π using the Chudnovsky formula, one of the fastest known infinite series for calculating π with very high precision. This formula involves computing large factorials and exponentiations, which require extensive floating-point operations and put heavy load on the CPU's arithmetic logic unit and floating-point unit. Each iteration of the series contributes around 14 digits of π, and the benchmark counts how many such iterations the CPU can complete in a fixed time interval. The result ("iterations per second") reflects the raw computational throughput of a single CPU core.

```
Algorithm CPU_SingleCore_PiComputation (iterations):

 long double sum = 0.0;
for (int k = 0; k < iterations; k++) {
   long double num = (long double)(factorial(6*k)) *
              (13591409.0 + 545140134.0 * k);
   long double den = (long double)(factorial(3*k)) *
              pow(factorial(k), 3) *
              pow(-640320.0, 3*k);
   sum += num / den;
}
long double pi = (426880.0 * sqrtl(10005.0)) / sum;
return pi;
```

IV. CPU multi-core – measures aggregate performance

This test measures the device's overall parallel processing capability by distributing a computationally heavy workload across all available CPU cores. Unlike the single-core test (which repeatedly computes π digits), this version uses parallel matrix multiplication, a classic CPU-bound algorithm that stresses arithmetic throughput, memory bandwidth, and thread-scheduling efficiency.

Each CPU core processes a distinct block of matrix rows, and the final performance score reflects the total execution time for all threads running concurrently. Faster devices with efficient scaling will complete the matrix multiplication significantly quicker.

```
Algorithm CPU_MultiCore_MatrixMultiply(size, thread_count)
allocate matrixA[size][size]
allocate matrixB[size][size]
allocate result[size][size]

rows_per_thread = size / thread_count

spawn thread_count threads:
  each thread T handles rows:
    start_row = T.id * rows_per_thread
    end_row   = (T.id is last)? size : start_row + rows_per_thread

  for i = start_row → end_row - 1:
    for j = 0 → size - 1:
      sum = 0
      for k = 0 → size - 1:
        sum += matrixA[i][k] * matrixB[k][j]
      result[i][j] = sum

wait for all threads to finish

free all matrices

return execution_time_in_seconds
```

V. GPU – rendering of a fixed 3D scene

The GPU benchmark evaluates the device's graphics performance by rendering a fixed 3D scene using OpenGL ES 2.0. The test creates an off-screen GLSurfaceView, initializes a renderer, and continuously draws multiple rotating cubes for a fixed duration.

Each rendered frame is timestamped, and the time between frames is used to calculate:

- Average FPS
- Minimum & Maximum FPS
- 95th percentile FPS (P95)
- 99th percentile FPS (P99)
- Total frame count

The test stresses both the vertex pipeline (through many model transformations) and the fragment pipeline (through multiple colored surfaces). The result approximates real-world GPU responsiveness under sustained rendering load.

```
Algorithm GPU_Render_Test(durationSeconds):
  create hidden GLSurfaceView
  set EGL context to OpenGL ES 2.0
```

```
  attach BenchmarkRenderer
  start continuous rendering

  wait until renderer.surfaceReady == true
  renderer.startBenchmark(durationSeconds)

  while renderer.isComplete() == false:
    sleep(100 ms)

  results = renderer.getResults()
  remove GLSurfaceView
  return results

On each frame:
  now = current time
  frameTime = now – lastFrameTime

  if frameTime is valid:
    frameTimes.append(frameTime)
  lastFrameTime = now

  if elapsed time >= benchmarkDuration:
    isRunning = false
    completed = true

  clear color & depth buffers
  set camera matrix

  for each cube in 5x10 grid:
    compute model matrix (translate + rotate)
    compute MVP matrix
    drawCube()

  rotation += constant value
```

VI. Memory – measures the bandwidth

This test measures how fast the device can transfer data in RAM. A large memory buffer (100 MB) is repeatedly copied from one region to another for a fixed duration. By counting the number of bytes copied per second, the test estimates the memory bandwidth of the device's RAM and memory controller.

```
Algorithm Memory_Bandwidth_Test(duration, buffer_size):
  source = allocate(buffer_size)
```

```
    dest   = allocate(buffer_size)
    fill(source, 0xAA)
    start_time = current_clock()
    end_time   = start_time + duration_seconds
    bytes_copied = 0

    while current_clock() < end_time:
      copy buffer_size bytes from source → dest
      bytes_copied = bytes_copied + buffer_size

    end_time_actual = current_clock()
    elapsed_seconds = end_time_actual - start_time
    bandwidth_GBps = (bytes_copied / elapsed_seconds) / (1024^3)
    free(source)
    free(dest)
    return bandwidth_GBps
```

VII. I/O (Storage) – random and sequential throughput measuring

This test evaluates device storage speed by writing data blocks sequentially to a temporary file. It measures how many bytes can be written per second, showing the real throughput of the internal flash storage.

```
Algorithm IO_Sequential_Write(duration, block_size):
  temp_file = open_new_file()
  bytes_written = 0

  start_time = current_time()

  while current_time() - start_time < duration:
    block = create_block(block_size)
    write_to_file(temp_file, block)
    bytes_written = bytes_written + block_size

  flush_to_storage(temp_file)
  close_file(temp_file)

  throughput = bytes_written / duration

  return {
    "bytes_written": bytes_written,
    "throughput_Bps": throughput
  }
```

4. DESIGN

4.1 OVERVIEW

This chapter defines the overall structure of the Android benchmark application, explaining how the performance evaluation concepts described in Analysis are used to design the system and present the general architecture, the components and the interactions between them.

4.2 USE OF THEORETICAL CONCEPTS

The performance parameters presented in Analysis (CPU throughput, GPU rendering, Memory latency, Storage I/O speed) are each represented by individual benchmark modules in the application. The measurement, composed of workload generation, test execution, data collection and result interpretation, is reflected in the internal workflow of the application.

4.3 GENERAL SYSTEM ARCHITECTURE

Figure 4.1 shows the general system architecture of the application. The system is structured around the Controller that coordinates the execution of individual tests selected by the user (CPU, GPU, Memory, Storage). Each test module generates a specific workload and reports the performance data to the Result Aggregator, which processes and sends it to the Data Visualization component. The User Interface component interacts only with the Controller and the Data Visualization component.

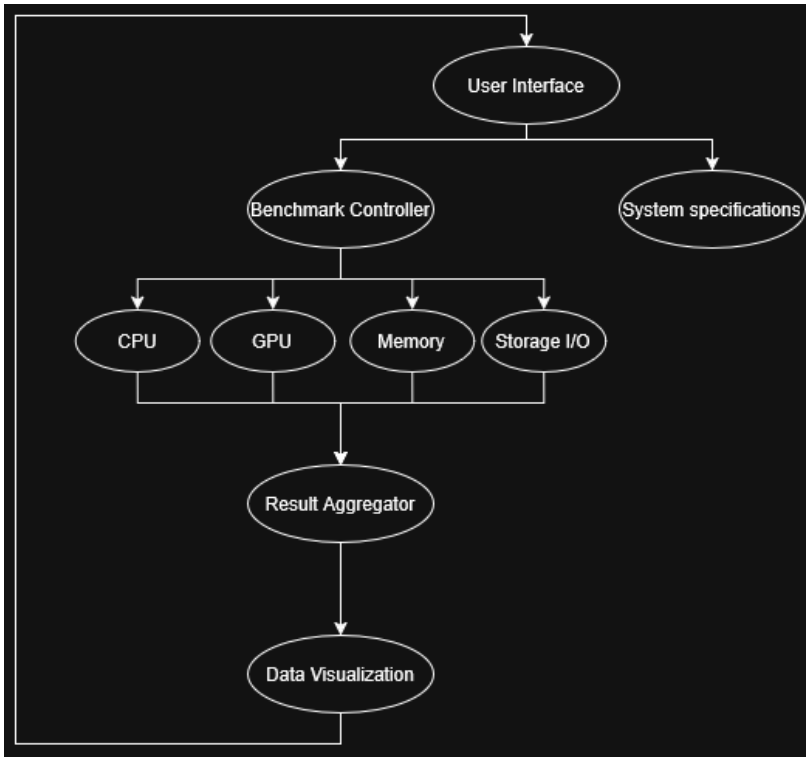

Figure 4.1 General System Architecture

4.4 UML ARCHITECTURE

I. User Actions

Figure 4.2 shows the actions the user can perform inside the application. It highlights the main interactions available, such as selecting which tests to run, starting a test run with the desired selection, and viewing the results. The diagram gives a quick overview of what the user can do while using the system.

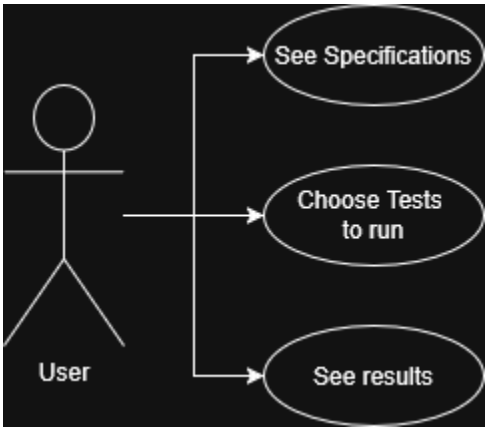

Figure 4.2 Use Case Diagram

II. Structure of classes

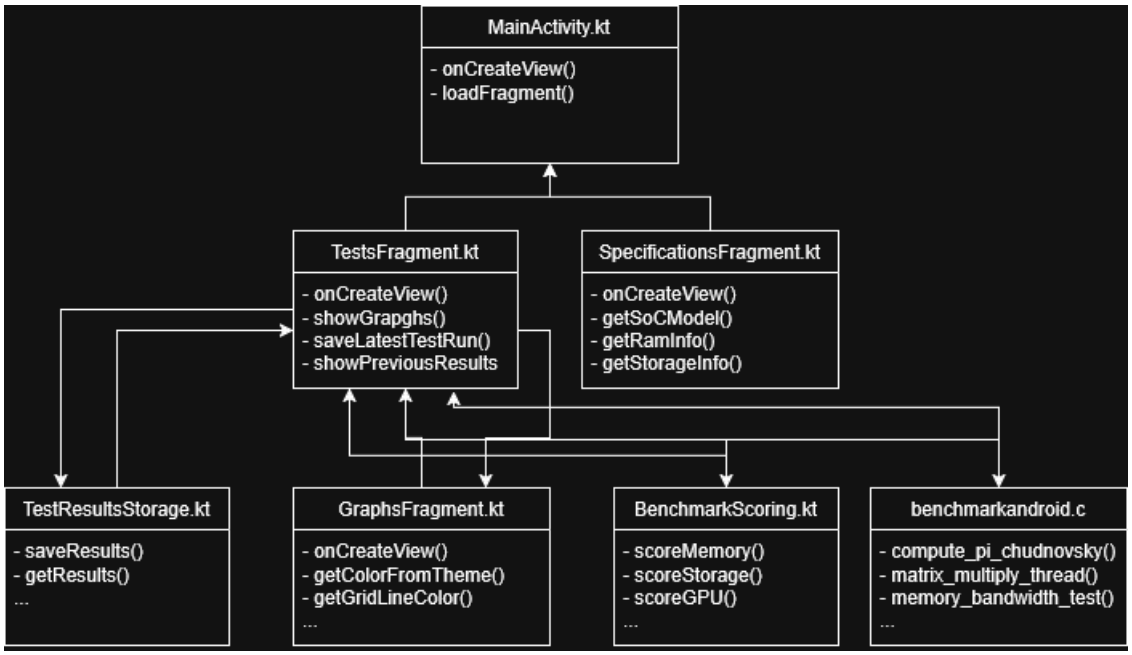

Figure 4.3 Structure of classes used inside the application

4.5 COMPONENTS GENERAL DESCRIPTION

I. Controller

Manages the executions of the user selected tests, ensures synchronization between modules and initializes the performance measurement process.

II. CPU module

Generates arithmetic, logic and multi-threaded workloads to evaluate the performance of the CPU.

III. GPU module

Renders a predefined 3D scene in order to evaluate the performance of the GPU.

IV. Memory module

Measures latency and bandwidth for reading and writing operations.

V. I/O Storage module

Measures Read/Write throughput and operations per second.

VI. Result Aggregator

Collects data from the test modules, analyses and computes final scores which it will send to the user interface.

VII. User Interface

Provides access to the functionality of the system by allowing the user to select which tests to run and display the scores and specifications of the system.

4.6 Component Interaction

When the user initiates a test, the **User Interface** signals the **Controller** which activates the corresponding test module. Once the execution is complete the **Test Module** sends data to the **Result Aggregator**. The computed scores are then sent back to the **User Interface** for visualization.

5. Implementation

5.1 Implementation Overview

The development of the application was carried out in two main parts inside the Android Studio IDE.

The first part focused on implementing the benchmarking tests themselves, integrating the native C code and ensuring that each algorithm ran correctly on the device. This included verifying numerical accuracy, checking execution times, and making sure the JNI layer worked reliably across different hardware.

The second part involved building the user interface, creating the structure of the app and ensuring smooth navigation between screens. During this phase the project was also polished overall—fixing bugs, resolving inconsistencies between components, and improving the visual layout. This combined effort ensured that the final application feels stable, coherent, and pleasant to use, with a clean interface and responsive interactions.

5.2 Phase 1: Benchmarking Tests

The first phase of the implementation process focuses on the benchmarking tests that will be executed by the application. These tests cover all major hardware components and provide a balanced overview of the device's computational capabilities. Specifically:

- a test that computes π using the Chudnovsky formula focusing on the single-core performance of the CPU
- a test that computes parallel matrix multiplication, each CPU core processing a distinct block of matrix rows, thus showing the capabilities of the CPU in multi-core tasks

- a test that renders a fixed 3d scene that continuously draws multiple rotating cubes for a fixed duration, determining the performance of the phone's GPU
- a test that measures the speed with which the device can transfer data in RAM, by repeatedly copying a large memory buffer (100 MB) from one region to another for a fixed duration
- a test that evaluates the device's storage speed by writing data blocks sequentially to a temporary file

Together, these benchmarks form a comprehensive suite that stresses different subsystems under controlled workloads. Each test was designed to be repeatable, deterministic, and independent of external factors, ensuring that the measured performance reflects the device's actual hardware capabilities as closely as possible. This phase also involved defining clear execution rules, handling timing accuracy, managing background threads, and ensuring that the tests run reliably across different Android versions and hardware architectures.

To convert the raw performance measurements into comparable results, the application includes a dedicated scoring module (BenchmarkScoring). Its purpose is to normalize the different test outputs—time, bandwidth, FPS—into a unified 0–1000 point scale. This allows devices with very different hardware characteristics to be compared using a single aggregated score.

Each benchmark category uses a simple linear model:

- **CPU single-core & multi-core tests**: these are time-based tests, where a lower execution time indicates better performance. The score is computed by comparing the measured time against a reference baseline. A device that completes the test in the baseline time receives 1000 points, while slower devices receive proportionally fewer points, down to a minimum of 0.

- **Memory benchmark**: this test measures RAM bandwidth. Higher bandwidth results in a higher score. The model assumes a reference peak throughput (e.g., 50 GB/s), assigning 1000 points to devices that reach or exceed it.

- **Storage benchmark**: the storage score is based on sequential write throughput. Faster internal storage yields higher scores, with 1000 points corresponding to the reference maximum performance.

- **GPU benchmark**: the GPU score is derived from the average FPS achieved during the 3D rendering test. The scoring scales linearly with FPS up to a cap of 1000 points, preventing extremely fast devices from skewing comparisons.

- **Overall score**: after all tests are completed, the overall device score is computed as the arithmetic mean of the individual test scores. This keeps the scoring fair and prevents any single benchmark from dominating the final result.

This scoring system makes the raw measurements easier to interpret and supports consistent comparisons between devices, regardless of differences in hardware capabilities.

There is also a Test Result Saving Module, which handles storing the outputs generated by each benchmark. After a test finishes, the module collects the measured values and writes them to

a simple text file inside the app's internal storage. This ensures that users can access past benchmark results even after closing the application, providing continuity and allowing basic comparison between runs without requiring external storage permissions or cloud sync.

5.3 PHASE 2: USER INTERFACE

The second phase is focused on building the structure of the Android application. The app is organized around a **MainActivity**, which hosts two main screens: the **SpecificationsFragment**, where the device's system information is displayed, and the **TestsFragment**, where the user can select which benchmark to run and can also see the statistics of the last 5 test runs. Navigation between these screens is handled through a **bottom navigation bar** with two buttons, allowing the user to switch quickly between viewing system specs and choosing a test. During this phase, the layout files were created, the fragments were connected to the activity, and the basic UI flow was established to ensure the application feels simple and intuitive to use.

Figure 5.1 shows the specifications tab on a Samsung S24 Ultra with the latest version of Android (Android 16). This is the default tab displayed when the app starts. Users can switch between this tab and the Tests tab as needed. The Specifications tab provides a clear and concise summary of the device's components, including the device name, SoC (System on a Chip) model, CPU architecture, number of CPU cores along with their current and maximum frequencies, GPU vendor and renderer, display resolution, total and available RAM and storage, and the Android version running on the smartphone.

Figure 5.2 shows the application's Tests tab, this is where the user can select the test they want to run, after having selected at least one test the *Start Tests* button becomes active and can be pressed to run the selected tests. Next to this button to the left there is a *Previous Tests* button which allows the user to see the last 5 test runs performed with results and scores. To the right of the *Start Tests* button there is a *Show Graphs* button which can be pressed to show the results of the latest test run in a graphic format.

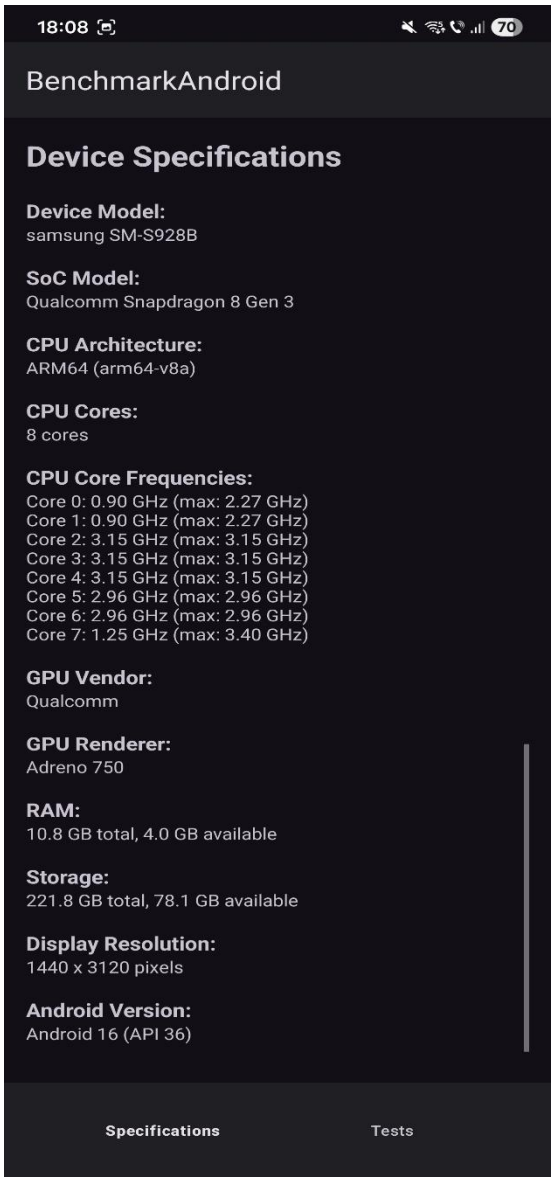

Figure 5.1 Specifications Tab

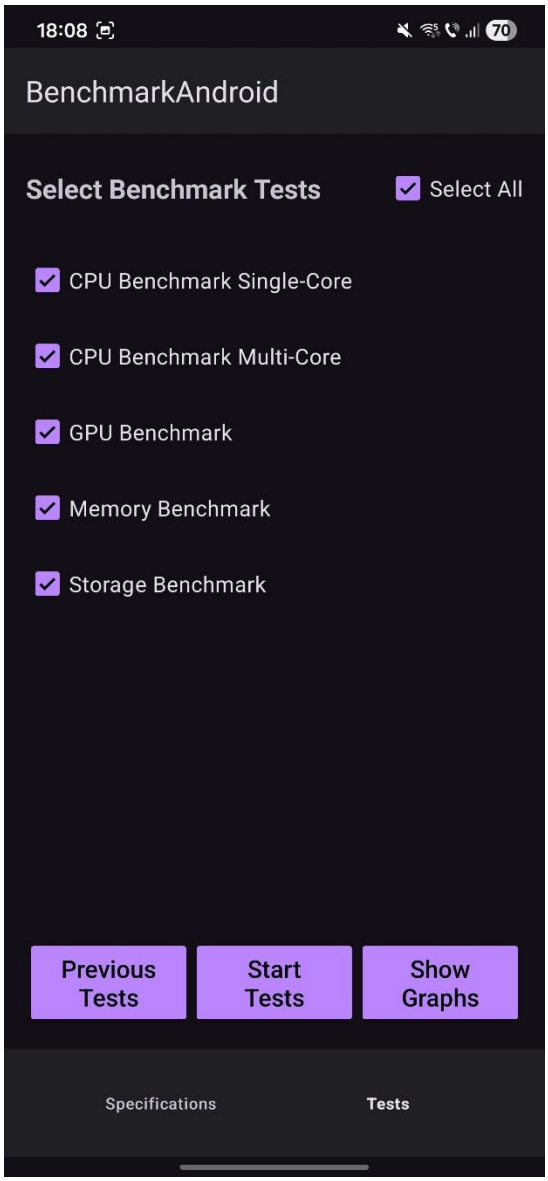

Figure 5.2 Tests Tab

Figure 5.3 shows the graphs screen, here the user can see the difference between the algorithm test runs that were computed during the testing phase, and in the case of the GPU test, the user can see the average FPS the device maintains each second while the 3D rendering is running.

Figure 5.4 shows the statistics screen, which becomes available only if at least one test has been performed, otherwise it is inactive. This data persists indefinitely even if the application is closed, and it also gives an overall score at the very top which is achieved by computing the average overall score of the last 5 complete test runs which aims to provide the user with the most accurate approximation of the device's performance.

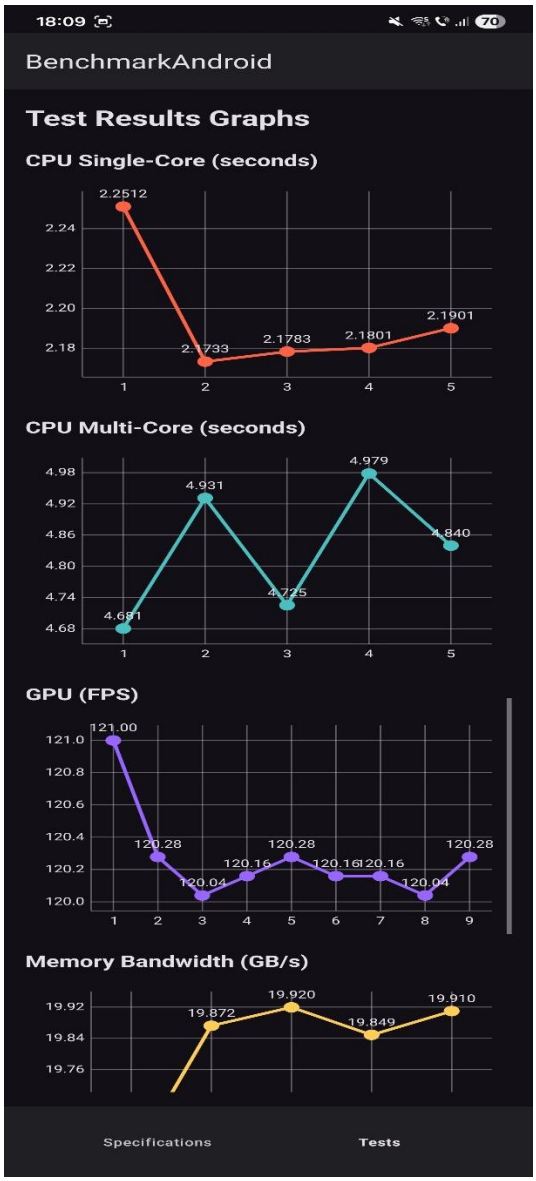

Figure 5.3 Graphs Screen

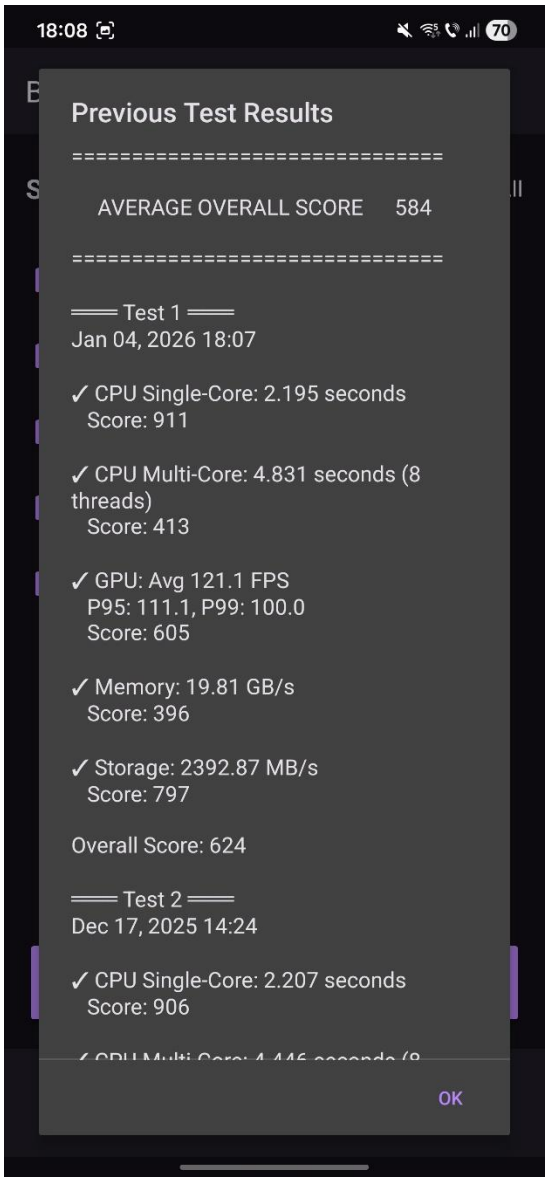

Figure 5.4 Statistics screen

6. Testing and Validation

The testing and validation phase aimed to verify the correctness, stability, and consistency of the benchmarking application across different hardware configurations and Android versions. To ensure meaningful results, the application was tested on two real devices with significantly different performance characteristics:

- **Samsung Galaxy S24 Ultra**, running **Android 16**
- **OnePlus 7T**, running **Android 11**

Using devices from different generations and Android versions allowed the evaluation of both backward compatibility and scalability on modern hardware.

In *Figure 6.1* we can see the results and scores of the S24 Ultra, and in *Figure 6.2* the results and scores of the OnePlus 7T. As expected, the difference in performance between the 2 devices is very noticeable, with an overall score of 636 compared to 275. Very noticeable differences occur in the following tests:

- ***GPU test:*** The S24 Ultra more than doubled the average FPS the OnePlus 7T could maintain, and a score of 605 to 300. This is expected seeing as the newer device has an Adreno 750 GPU and the older device an Adreno 640
- ***Memory test:*** Again, the S24 Ultra has more than doubled the memory bandwidth capacity of the OnePlus 7T, with an average speed of 21.72 GB/s to 10.61 GB/s and the scores of 434 to 212
- ***Storage test:*** Here we can see the biggest difference between the 2 devices, the S24 Ultra having a throughput of 2450.91 MB/s to the 490.24 MB/s of the OnePlus 7T, a difference of almost 5 times. The scores are 816 to 163

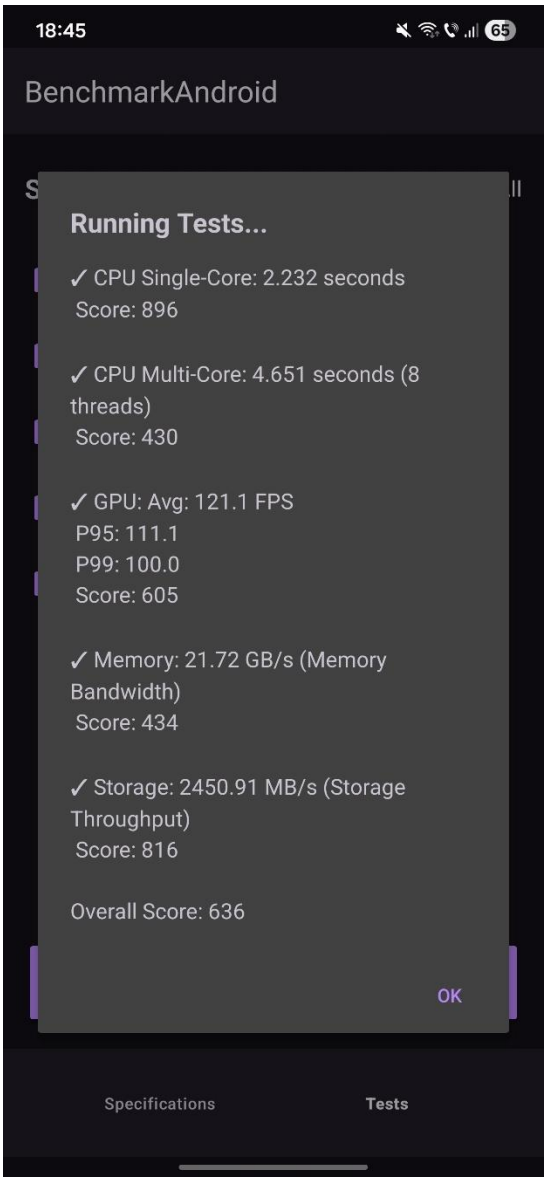

Figure 6.1 Test Results on S24 Ultra

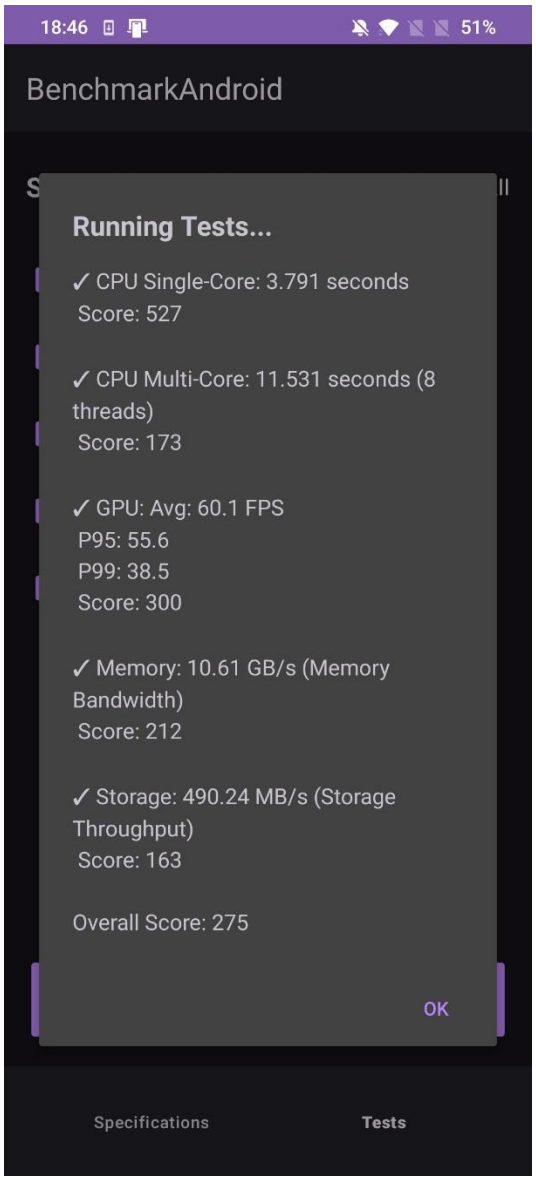

Figure 6.2 Test Results on OnePlus 7T

Running the application on Android 11 and Android 16 confirmed that the app functions correctly across different Android API levels. Differences in system behavior, such as thread scheduling and power management, did not affect the correctness of the results.

After having run all of the test runs 5 times to get a more accurate overall score, we get the following: the S24 Ultra has an average overall score of 639 and the OnePlus 7T an average overall score of 276. Minor variations were observed between individual runs, which is expected due to background system activity and thermal management; however, these variations did not significantly affect the final average scores.

These results confirm that the benchmarking application produces stable, repeatable, and hardware-representative measurements, making it suitable for comparative performance evaluation across different Android devices.

7. Conclusions

This project successfully achieved its goal of designing and implementing an Android benchmarking application capable of evaluating the performance of key hardware components in a mobile device. By combining CPU, GPU, memory, and storage benchmarks into a unified testing suite, the application provides a clear and comprehensive overview of a device's computational capabilities.

The implemented benchmarks were carefully chosen to reflect realistic workloads, such as intensive mathematical computations, parallel processing, 3D rendering, memory transfers, and sequential storage access. The results obtained from these tests were normalized using a consistent scoring model, allowing meaningful comparisons between devices with different hardware configurations.

Testing on two devices from different generations, the Samsung Galaxy S24 Ultra and the OnePlus 7T, demonstrated that the application behaves correctly across multiple Android versions and hardware platforms. The significant performance differences observed between the two devices align with their expected hardware capabilities, validating both the correctness of the implemented algorithms and the reliability of the measurement methodology.

Furthermore, repeated test executions confirmed that the application produces stable and repeatable results, even under prolonged use. The user interface was designed to be intuitive and responsive, allowing users to easily navigate between system specifications, benchmark execution, and result visualization.

8. References

[1]. Geekbench Official Website - https://www.geekbench.com/ [October 2025]

[2]. Preview to AnTuTu Benchmark & 3DRating V4.0 - https://www.antutu.com/en/doc/105870.htm [October 2025]

[3]. 3DMark Section on the Benchmarks.ul Website – https://benchmarks.ul.com/3dmark [October 2025]